9 Collections 1 – Tuples and Lists

Tuples First

Create a program called **tuples_first**. The program should first input an integer and a float from the user. Then the program should create a tuple that holds the inputted two values. Finally, the program should print the tuple.

Example output:

```
Enter an int: 5
Enter a float: 0.29
(5, 0.29)
```

Tuples Argument

Create a program called **tuples_argument**. The program should have a function named **print_player** that takes a tuple as argument. The tuple should contain a team name, player name and the number of the goals the player has scored. Finally, the **print_player** function should print these values.

Example output:

```
Jennifer (Hawks), 10 goals
```

First, copy/paste the *main function* below to your program. Then write the **print_player** function.

```
def main():
    p = ("Hawks", "Jennifer", 10)
    print_player(p)

main()
```

NB! In *some exercises* in this exercise set, there are unit tests in Viope, which will test only your function instead of testing your whole program. Therefore, there are no example outputs given for those exercises.

Tuples Return

Create a program called **tuples_return**. The program should have a function named **pow_2_3** that takes an integer as argument. The function should compute the second power and the third power of the integer and return them in a tuple.

First, copy/paste the *main function* below to your program. Then write the **pow_2_3** function.

```
def main():
    x = int(input("Enter an integer: "))
    p2, p3 = pow_2_3(x)
    print(p2)
    print(p3)

if __name__ == "__main__":
    main()
```

Lists Remove Min Max

Create a program called **lists_remove_min_max**. The program should have a function named **remove_min_max** that takes a list as argument. The function should remove the smallest and the largest value in the list. Note: The function should be able to handle an empty list and a list that has only one element.

First, copy/paste the *main function* below to your program. Then write the **remove_min_max** function.

```
def main():
    a = [3, 1, 4, 1, 5]
    remove_min_max(a)
    print(a)

if __name__ == "__main__":
    main()
```

Tuples Entry Exam

Create a program called **tuples_entry_exam**. The program should have a function named **grade_exam** that takes a list of applicants and the exam passing score as arguments. In the list, applicant names and exam scores are contained in tuples.

The **grade_exam** function should return a new list of tuples that contains only those applicants' names and scores who have passed the entry exam.

First, copy/paste the *main function* below to your program. Then write the **grade_exam** function.

```
def main():
    applicants = [("Ann", 30), ("Jack", 25), ("Jill", 40)]

    passed_applicants = grade_exam(applicants, 30)
    print("Entry exam passed")
    for name, points in passed_applicants:
        print(f"{name}, {points} points")

if __name__ == "__main__":
    main()
```

Tuples Swap

Create a program called **tuples_swap.** The program should first input seven integers from the user and save them in a list. Then the program should swap each successive pair of elements in the list. Finally, the program should print the list as shown

Please see the example output for more details.

NB! You should use **slicing** when you swap a pair of elements.

Example output:

```
Enter an integer: 1
Enter an integer: 2
Enter an integer: 3
Enter an integer: 4
Enter an integer: 5
Enter an integer: 6
Enter an integer: 7
[2, 1, 4, 3, 6, 5, 7]
```

Tuples Clock

Create a program called **tuples_clock.** The program should have a function named **roll_forward**. The function should take the following as arguments:

- clock time as a tuple that contains hours (int) and minutes (int)
- the number of hours to be added to the clock time
- the number of minutes to be added to the clock time.

The **roll_forward** function should *roll the clock time forward* and return a new clock time as a tuple. In the new clock time, hours should be between 0 and 23, and minutes should be between 0 and 59.

The clock time should start from 00:00. The ***main function*** should

- input hours and minutes to be added to the clock time
- call the roll_forward function to roll the time forward
- print the new clock time.

The program should terminate when the user enters negative hours.

Example output:

```
The current time is 00:00
Enter hours to add: 12
Enter minutes to add: 30
12:30
Enter hours to add: 5
Enter minutes to add: 35
18:05
Enter hours to add: 0
Enter minutes to add: 1439
18:04
Enter hours to add: -1
```

NB! You are not required to validate any input values or arguments in this program.